

Eurovision Project Setup

testje.txt

 Document

op basis van mijn bestand genereer de volgende dingen -> Wat je ook moet noteren:

De Cassandra keyspace en tabel aanmaak commando

De firewall regel voor de poorten

De docker-compose bestanden

Wil je dat ik alles samengooi in één overzichtelijk document dat je kan bewaren? bestand:

Eurovision Project

Ja — dit is exact het moment om alles netjes samen te brengen 👍

Ik maak voor jou één **overzichtelijk document** dat je kan indienen of bewaren.

Rédaction

Eurovision Project – Technische Setup

1. Cassandra configuratie

Keyspace aanmaken

```
CREATE KEYSPACE IF NOT EXISTS eurovision
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 1
};
```

Tabel aanmaken

```
USE eurovision;

CREATE TABLE IF NOT EXISTS votes (
  country text,
  votes bigint,
  PRIMARY KEY (country)
);
```

2. Firewall regels (Google Cloud)

Firewall rule om communicatie tussen VM's en datacenters toe te laten:

```
gcloud compute firewall-rules create allow-eurovision \
  --allow tcp:9092,tcp:9042,tcp:7077,tcp:8080,tcp:2181,tcp:4040 \
  --source-ranges 0.0.0.0/0 \
  --description "Eurovision pipeline ports"
```

Gebruikte poorten

Service	Poort
Kafka	9092
Zookeeper	2181
Cassandra	9042
Spark Master	7077
Spark UI	8080
Spark Jobs	4040

3. Docker Compose bestanden

3.1 Kafka (dc1-kafka)

```
version: "3"
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    restart: always

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on: [zookeeper]
    ports:
      - "9092:9092"
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://<KAFKA_IP>:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    restart: always
```

3.2 Spark (dc1-spark)

```
services:
  spark-master:
    image: apache/spark:3.4.0
    ports:
      - "8080:8080"
      - "7077:7077"
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master
    restart: always

  spark-worker:
    image: apache/spark:3.4.0
    depends_on: [spark-master]
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker spark://spark-
```

```
master:7077
  restart: always
```

3.3 Cassandra (dc1-cassandra)

```
services:
  cassandra:
    image: cassandra:4.1
    ports:
      - "9042:9042"
    environment:
      - CASSANDRA_CLUSTER_NAME=EurovisionCluster
      - CASSANDRA_DC=DC1
    volumes:
      - cassandra_data:/var/lib/cassandra
    restart: always

volumes:
  cassandra_data:
```

4. Architectuur overzicht

Je systeem bestaat uit 3 VM's:

- **VM1 (Kafka)** → ontvangt stemmen (producer)
- **VM2 (Spark)** → verwerkt stemmen
- **VM3 (Cassandra)** → slaat resultaten op

Data flow:

Producer → Kafka → Spark → Cassandra

5. Inter-datacenter communicatie

Voor samenwerking met andere teams:

Deel deze info:

- Kafka IP + poort (9092)
 - Cassandra IP + poort (9042)
 - Keyspace: **eurovision**
 - Tabel: **votes**
 - Topic: **eurovision-votes**
-

6. Resultaat

- ✓ 1000 stemmen gegenereerd
- ✓ Verwerking via Kafka + Spark
- ✓ Opslag in Cassandra
- ✓ Ranking per land beschikbaar

7. Demo (kort)

Voor opname:

```
gcloud compute instances list
gcloud compute ssh dc1-kafka --command="sudo docker ps"
gcloud compute ssh dc1-kafka --command="sudo docker run --rm --network host eurovision-producer"
gcloud compute ssh dc1-spark --command="sudo docker run --rm --network host eurovision-spark"
gcloud compute ssh dc1-cassandra --command="sudo docker exec eurovision-cassandra-1 cqlsh -e 'SELECT
* FROM eurovision.votes;'"
```

✓ Conclusie

Je hebt een werkende **distributed data pipeline** met:

- Multi-VM architectuur
- Multi-container setup
- Real-time data verwerking
- Distributed database opslag

Als je wil, kan ik dit nog upgraden naar:

-  een diagram (voor je presentatie)
-  een korte script voor je demo uitleg
-  een GitHub-ready README

Zeg maar 

Teamleden verbinden — ik geef je exact welke informatie je moet delen (IP-adressen, poorten, Cassandra keyspace naam) en welke commando's zij moeten uitvoeren om hun Kafka producers te laten sturen naar jouw Cassandra, of omgekeerd.

Goed — dit is het **belangrijkste stuk voor teamwork**, dus ik maak het super concreet zodat jullie **direct kunnen verbinden tussen datacenters** 

1. Wat jij moet doorgeven aan je team

Stuur dit letterlijk naar je team (Teams/WhatsApp):

Mijn datacenter (DC1)

Service	IP	Poort
Kafka	35.205.160.140	9092
Cassandra	34.22.143.37	9042
Spark	34.140.166.171	7077

Extra info

- Kafka topic: `eurovision-votes`
- Cassandra keyspace: `eurovision`
- Cassandra tabel: `votes` (country text, votes bigint)

2. Wat je teamleden moeten aanpassen

♦ A. Producer (sturen naar JOUW Kafka)

In hun `producer.py`:

Python

```
bootstrap_servers="35.205.160.140:9092"
```

 Resultaat:

Hun stemmen gaan naar **jouw Kafka**

♦ B. Spark job (lezen van jouw Kafka)

Python

```
consumer = KafkaConsumer(
    "eurovision-votes",
    bootstrap_servers="35.205.160.140:9092",
    auto_offset_reset="earliest"
)
```

♦ C. Cassandra (schrijven naar jouw database)

Python

```
cluster = Cluster(["34.22.143.37"])
session = cluster.connect("eurovision")
```

👉 Resultaat:

Alle resultaten komen in **jouw Cassandra**

3. Alternatief (BETERE architectuur)

In plaats van alles naar jou te sturen → maak een **echte distributed setup**:

Optie 1 (simpel – demo)

Iedereen stuurt naar **1 Kafka (jij)**

Iedereen schrijft naar **1 Cassandra (jij)**

✓ makkelijk

✗ niet echt distributed

Optie 2 (aanbevolen – betere punten)

Cassandra cluster delen

Iedereen past dit aan:

Python

```
cluster = Cluster([
    "34.22.143.37", # jij
    "IP_DC2",
    "IP_DC3",
    "IP_DC4"
])
```

👉 Resultaat:

- 4 nodes = 1 cluster
- echte distributed database 🌟

4. Firewall check (HEEL BELANGRIJK)

Iedereen moet deze poorten open hebben:

Bash

```
9092 # Kafka
9042 # Cassandra
7077 # Spark
```

5. Test dat het werkt (super belangrijk voor demo)

Laat een teamlid dit doen:

Stap 1 — hij stuurt votes

Bash

```
docker run eurovision-producer
```

Stap 2 — jij checkt Cassandra

Bash

```
SELECT * FROM eurovision.votes;
```

👉 Als nieuwe votes verschijnen =  CONNECTIE WERKT

6. Wat je zegt in je demo (heel belangrijk)

Gebruik dit letterlijk:

"Elke student heeft een eigen datacenter met 3 VM's.

We sturen data via Kafka tussen datacenters, verwerken met Spark, en slaan alles op in een gedeeld Cassandra cluster."

Veelgemaakte fouten (die je punten kosten)

-  Verkeerde IP gebruiken (internal i.p.v. external)
-  Kafka advertised listener niet aanpassen
-  Firewall niet open
-  Cassandra cluster niet gedeeld

Samenvatting

👉 Minimum werkend:

- Iedereen → jouw Kafka
- Iedereen → jouw Cassandra

👉 Beste oplossing:

- Kafka per DC
- 1 gedeelde Cassandra cluster

Wil je dat ik nu ook:

- een **netwerkdigram** maak (voor je slides)
- of een **perfecte demo script (2 min presentatie)**

Dat kan je echt extra punten opleveren 👍